MLOps – Assignment : 01

Name :	Harshitha S
USN :	4GW23CI016-CSE(AI-ML)
Date :	25/09/2026

Objective :

The objective of this assignment is to process a sales dataset using Apache Pig on a Hadoop environment. **The assignment performs the following tasks:**

- Start the Hadoop HDFS and YARN services.
- Check that all Hadoop daemons are running using `jps`.
- Create a folder in HDFS and upload a CSV sales dataset into it.
- Write an Apache Pig script to read and clean the dataset.
- Find the total number of orders and the total sales amount for each city.
- Sort the cities from highest to lowest total sales.
- Store the final result in HDFS and display it.

Dataset :

The input dataset is a CSV file named **sales.csv** stored in HDFS at:

- **/user/aiml/sales/sales.csv**

The dataset contains three fields:

- **Order ID** – Unique number for each order.
- **City** – City where the order was placed.
- **Amount** – Sale amount of the order (in ₹).

INPUT :

Order ID	City	Amount (₹)	Status
101	Bangalore	1200	Valid
102	Mysore	850	Valid
103	Bangalore	0	*Removed (zero)*
104	Mangalore	640	Valid
105	Mysore	–	*Removed (missing)*
106	Hubli	450	Valid
107	Bangalore	980	Valid
108	Mangalore	-50	*Removed (negative)*
109	Mysore	700	Valid
110	Hubli	1250	Valid
111	Mangalore	900	Valid
112	Bangalore	500	Valid

Pig Script :

The Apache Pig program loads the sales dataset from HDFS, removes invalid orders, groups the remaining orders by city, counts the orders and adds up the sales amount for each city, sorts the result, and stores it back in HDFS. The overall flow of the script is shown below.

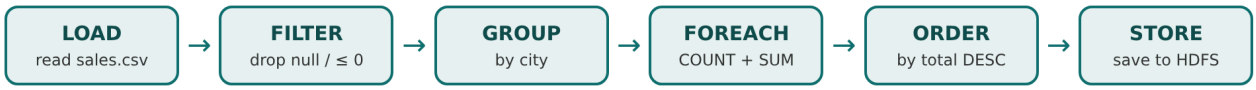


Fig 1 : Data flow of the Pig script

```
%%writefile sales.pig
-- Load sales data from HDFS
sales = LOAD '/user/aiml/sales/sales.csv'
        USING PigStorage(',')
        AS (order_id:int, city:chararray, amount:int);

-- Remove missing, zero and negative amounts
valid_sales = FILTER sales BY amount IS NOT NULL AND amount > 0;

-- Group valid orders by city
grouped_sales = GROUP valid_sales BY city;

-- Count orders and add up sales for each city
city_totals = FOREACH grouped_sales
              GENERATE group AS city,
                       COUNT(valid_sales) AS total_orders,
                       SUM(valid_sales.amount) AS total_sales;

-- Sort cities from highest to lowest sales
sorted_totals = ORDER city_totals BY total_sales DESC;

-- Store the result in HDFS
STORE sorted_totals INTO '/user/aiml/sales/output'
USING PigStorage(',');
```

Fig 2 : Pig script – sales.pig

Code Explanation :

The **LOAD** operation reads the sales.csv file from HDFS. `PigStorage(',')` tells Pig that the values in each line are separated by commas. The schema names the three columns: **order_id** as an integer, **city** as a character string, and **amount** as an integer.

The **FILTER** operation removes bad records. The condition `amount IS NOT NULL AND amount > 0` keeps only orders that have a proper positive amount. So order 103 (amount 0), order 105 (missing amount) and order 108 (amount -50) are removed before any calculation.

The **GROUP** operation collects the valid orders city-wise. All Bangalore orders go into one group, all Mysore orders into another, and the same for Hubli and Mangalore.

The **FOREACH … GENERATE** operation works on each city group one by one. `COUNT()` counts how many valid orders the city has, and `SUM()` adds up their amounts. The city name, order count and total sales are generated as one output row.

The **ORDER BY … DESC** operation sorts the cities so that the city with the highest total sales comes first.

Finally, the **STORE** operation saves the sorted result in the HDFS folder **/user/aiml/sales/output** in comma-separated format using `PigStorage(',')`.

Execution :

The Hadoop services were started, the dataset was uploaded to HDFS, and the Pig script was run in MapReduce mode using the following commands:

```
start-dfs.sh
start-yarn.sh
jps
hdfs dfs -mkdir -p /user/aiml/sales
hdfs dfs -put sales.csv /user/aiml/sales/
hdfs dfs -cat /user/aiml/sales/sales.csv
hdfs dfs -rm -r /user/aiml/sales/output     (only if the output folder already exists)
pig -x mapreduce sales.pig
hdfs dfs -cat /user/aiml/sales/output/part-r-00000
```

Pig does not overwrite an existing output folder, so the old output folder is deleted before running the script again.



Fig 3 : Running the script and reading the result from HDFS

Output :

After successful execution, the result is stored in HDFS at **/user/aiml/sales/output/part-r-00000**. Each line contains the city, the number of valid orders and the total sales amount.

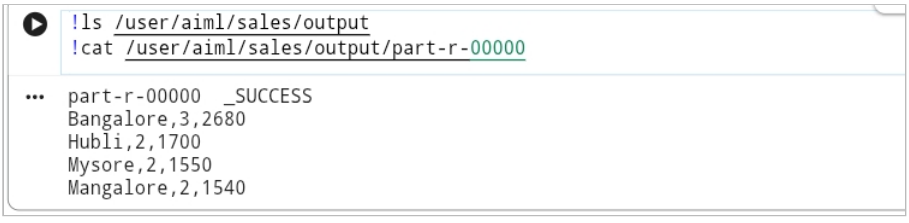


Fig 4 : Output file part-r-00000

City	Valid Orders	Calculation	Total Sales (₹)
Bangalore	3	1200 + 980 + 500	**2680**
Hubli	2	450 + 1250	**1700**
Mysore	2	850 + 700	**1550**
Mangalore	2	640 + 900	**1540**

Bangalore has the highest sales with 3 valid orders, since its zero-amount order was removed. Mysore has only 2 valid orders because one order had no amount. Mangalore also has 2 valid orders because the -50 entry was removed. Hubli had no invalid orders, so both its orders are counted.

Outcome :

The Apache Pig script was successfully executed in MapReduce mode on the Hadoop environment. The sales dataset was loaded from HDFS and records with missing, zero or negative amounts were filtered out. The valid orders were grouped by city, and the number of orders and total sales for each city were calculated using the COUNT() and SUM() functions. The result was sorted in descending order of total sales.

The final city-wise result was stored in HDFS. This assignment shows how Apache Pig can be used for loading, cleaning, grouping, aggregating, sorting and storing data in a Hadoop environment with only a few lines of script.